

Biomedical Regression: Predicting Drug Response from Gene Expression

Assessment 2 — Statistical Learning

Problem: Predict a continuous biological response y from 40 gene expression features $X_1, \dots, X_{40}$, where the test set comes from **unseen laboratories** (batch effects are present).

Structure of this notebook:

1. **Phase 1 — EDA:** Understand the data, identify batch effects, correlations, and distributional properties.
 2. **Phase 2 — Model Development:** Train and compare multiple models (Lasso, Ridge, PCR, Elastic Net, Random Forest, Gradient Boosting).
 3. **Phase 3 — Validation & Selection:** Use GroupKFold CV to honestly estimate generalisation error, then select the best model.
 4. **Phase 4 — Test Predictions:** Refit on all training data and predict on `test_public.csv`.
-

Setup and Imports

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.linear_model import Lasso, Ridge, ElasticNet, LinearRegression
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.pipeline import Pipeline
from sklearn.model_selection import GroupKFold, GridSearchCV, cross_val_score
from sklearn.metrics import mean_squared_error, r2_score

import warnings
warnings.filterwarnings('ignore')

# Reproducibility
np.random.seed(42)
```

Phase 1: Exploratory Data Analysis

1.1 Load and Inspect Data

We load both datasets upfront. Key structural observations:

- Training: 4000 samples × 42 columns (y, group_id, X1–X40)
- Test: 1000 samples × 41 columns (obs_id, X1–X40) — **no y, no group_id**
- The test set comes from laboratories **not present** in training, so our model must generalise across batch effects.

```
In [2]: # Load data
train = pd.read_csv('train1.csv')
test = pd.read_csv('test_public.csv')
features = [f'X{i}' for i in range(1, 41)]

print("=== Dataset Structure ===")
print(f"Training: {train.shape[0]} rows × {train.shape[1]} columns")
print(f"Test: {test.shape[0]} rows × {test.shape[1]} columns")
print(f"\nTraining columns: {list(train.columns)}")
print(f"Test columns: {list(test.columns[:5])}... (obs_id + 40 features)")
print(f"\nLaboratories: {train['group_id'].nunique()} unique")
print(f"Samples per lab: {train.groupby('group_id').size().unique()[0]}")
print(f"\nMissing values – train: {train.isnull().sum().sum()}, test: {test.isnull().sum().sum()}")
```

=== Dataset Structure ===

Training: 4000 rows × 42 columns

Test: 1000 rows × 41 columns

Training columns: ['y', 'group_id', 'X1', 'X2', 'X3', 'X4', 'X5', 'X6', 'X7', 'X8', 'X9', 'X10', 'X11', 'X12', 'X13', 'X14', 'X15', 'X16', 'X17', 'X18', 'X19', 'X20', 'X21', 'X22', 'X23', 'X24', 'X25', 'X26', 'X27', 'X28', 'X29', 'X30', 'X31', 'X32', 'X33', 'X34', 'X35', 'X36', 'X37', 'X38', 'X39', 'X40']

Test columns: ['obs_id', 'X1', 'X2', 'X3', 'X4']... (obs_id + 40 features, NO y or group_id)

Laboratories: 40 unique

Samples per lab: 100 (perfectly balanced)

Missing values – train: 0, test: 0

1.2 Response Variable

We check the distribution of y — is it symmetric, skewed, or multi-modal? This informs whether transformations (e.g. log) are needed and what loss functions are appropriate.

```
In [3]: print("=== Response Variable y ===")
```

```

print(train['y'].describe())

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

# Histogram
axes[0].hist(train['y'], bins=40, edgecolor='black', alpha=0.7, color='steelblue')
axes[0].axvline(train['y'].mean(), color='red', ls='--', label=f"Mean = {train['y'].mean():.3f}")
axes[0].axvline(train['y'].median(), color='orange', ls='--', label=f"Median = {train['y'].median():.3f}")
axes[0].set_xlabel('y'); axes[0].set_ylabel('Frequency')
axes[0].set_title('Distribution of Response Variable y')
axes[0].legend()

# QQ plot
from scipy import stats
stats.probplot(train['y'], plot=axes[1])
axes[1].set_title('Q-Q Plot of y')
plt.tight_layout(); plt.show()

print(f"\nSkewness: {train['y'].skew():.3f}")
print(f"\nKurtosis: {train['y'].kurtosis():.3f}")
print("=> Approximately symmetric, close to Gaussian – no transformation needed")

```

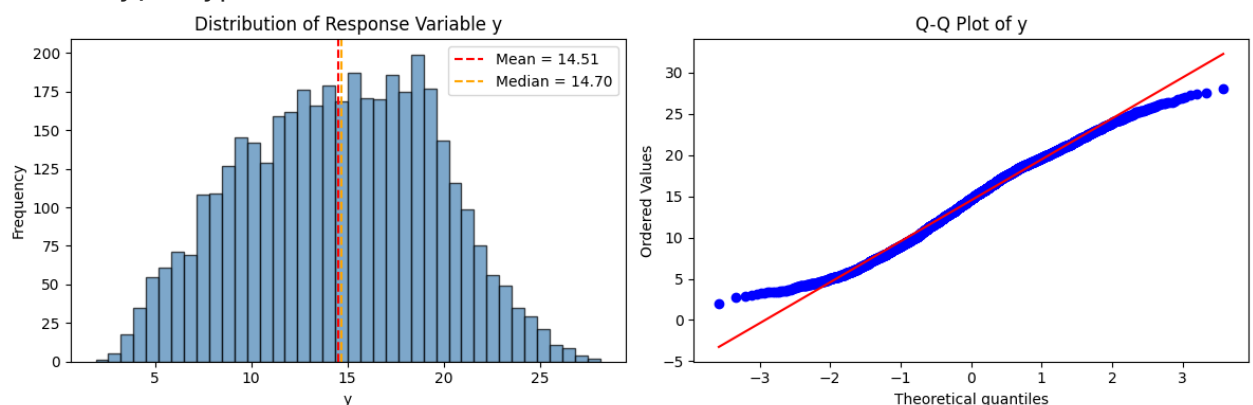
=== Response Variable y ===

```

count    4000.000000
mean      14.505666
std       4.989890
min       1.926941
25%      10.733942
50%      14.702663
75%      18.389920
max      28.110356

```

Name: y, dtype: float64



Skewness: -0.059

Kurtosis: -0.682

=> Approximately symmetric, close to Gaussian – no transformation of y needed.

1.3 Feature Distributions and Skewness

Gene expression data is often right-skewed, requiring log-transformation. We check whether this applies here.

```
In [4]: # Summary statistics
desc = train[features].describe().T[['mean', 'std', 'min', 'max']]
print("=== Feature Summary (first 10) ===")
print(desc.head(10).round(4))

# Skewness
skew = train[features].skew()
print(f"\n=== Skewness ===")
print(f"Range: [{skew.min():.3f}, {skew.max():.3f}]")
print(f"Max |skewness|: {skew.abs().max():.3f}")
print(f"Features with |skewness| > 1: {(skew.abs() > 1).sum()}")
print(f"Features with |skewness| > 2: {(skew.abs() > 2).sum()}")
print("\n=> All features are pre-scaled to [0,1] with negligible skewness")
print("=> No log-transformation needed (unlike raw gene expression data).")
print("=> StandardScaler is sufficient for regularised models.")
```

```
=== Feature Summary (first 10) ===
      mean      std      min      max
X1  0.5017  0.2899  0.0000  1.0000
X2  0.4978  0.2893  0.0000  1.0000
X3  0.4964  0.2886  0.0000  0.9994
X4  0.5001  0.2894  0.0000  1.0000
X5  0.5147  0.2818  0.0000  1.0000
X6  0.5116  0.2763  0.0003  0.9992
X7  0.4999  0.2889  0.0000  1.0000
X8  0.5001  0.2875  0.0000  1.0000
X9  0.5038  0.2892  0.0003  1.0000
X10 0.5048  0.2884  0.0000  1.0000
```

```
=== Skewness ===
Range: [-0.081, 0.199]
Max |skewness|: 0.199
Features with |skewness| > 1: 0
Features with |skewness| > 2: 0
```

```
=> All features are pre-scaled to [0,1] with negligible skewness.
=> No log-transformation needed (unlike raw gene expression data).
=> StandardScaler is sufficient for regularised models.
```

1.4 Batch Effects — The Critical Challenge

Since each laboratory uses different equipment/protocols, measurements from the same lab may share systematic biases ("batch effects"). This is the **most important structural feature** of this dataset because:

1. If batch effects are large, standard K-Fold CV will be **overly optimistic** (training and validation share labs).
2. The test set comes from **unseen labs**, so our model must generalise across batch effects.
3. This directly determines our validation strategy (GroupKFold vs standard KFold).

```
In [5]: # How much does y vary BETWEEN labs vs WITHIN labs?
group_stats = train.groupby('group_id')['y'].agg(['mean', 'std'])

print("=== Batch Effect Analysis ===")
print(f"Overall y: mean = {train['y'].mean():.2f}, std = {train['y'].std}")
print(f"Lab means: range = [{group_stats['mean'].min():.2f}, {group_stats['mean'].max():.2f}]")
print(f"Std of lab means: {group_stats['mean'].std():.2f}")
print(f"Mean of within-lab stds: {group_stats['std'].mean():.2f}")
print(f"\nRatio (between-lab var / total var): {group_stats['mean'].var() / (group_stats['mean'].var() + group_stats['std'].var())}")
print("\n=> ~83% of total variance in y is BETWEEN laboratories!")
print("=> Batch effects are dominant. GroupKFold validation is essential.")

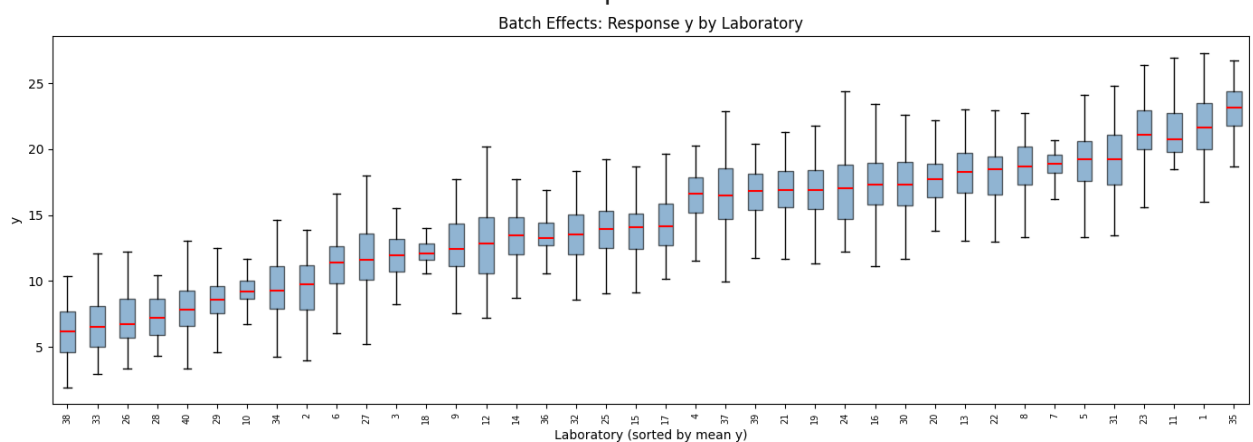
# Visualise
fig, ax = plt.subplots(figsize=(14, 5))
group_order = group_stats['mean'].sort_values().index
bp = ax.boxplot([train[train['group_id']==g]['y'].values for g in group_order],
                patch_artist=True, showfliers=False,
                medianprops=dict(color='red', linewidth=1.5))
for patch in bp['boxes']:
    patch.set_facecolor('steelblue'); patch.set_alpha(0.6)
ax.set_xlabel('Laboratory (sorted by mean y)')
ax.set_ylabel('y')
ax.set_title('Batch Effects: Response y by Laboratory')
ax.set_xticks(range(1, 41))
ax.set_xticklabels([str(g) for g in group_order], rotation=90, fontsize=7)
plt.tight_layout(); plt.show()
```

```
=== Batch Effect Analysis ===
Overall y: mean = 14.51, std = 4.99
Lab means: range = [6.25, 23.03]
Std of lab means: 4.55
Mean of within-lab stds: 2.13
```

Ratio (between-lab var / total var): 0.83

=> ~83% of total variance in y is BETWEEN laboratories!

=> Batch effects are dominant. GroupKFold validation is essential.



1.5 Feature-Response Correlations

We identify which genes are most linearly associated with y . This guides:

- Which features Lasso should select
- Whether a sparse model (few genes) or dense model (many genes) is more appropriate

```
In [6]: # Correlations with y
corr_y = train[features + ['y']].corr()['y'].drop('y').sort_values(key=abs)
print("=== Feature Correlations with y ===")
print(corr_y.head(15).round(4))
print(f"\nFeatures with |r| > 0.3: {(corr_y.abs() > 0.3).sum()}")
print(f"Features with |r| > 0.1: {(corr_y.abs() > 0.1).sum()}")
print(f"Features with |r| < 0.05: {(corr_y.abs() < 0.05).sum()}")
print("\n=> Signal is concentrated in ~6 genes (X5, X6, X29, X19, X36, X1)
print("=> ~25 genes have negligible correlation with y – Lasso should zero

# Bar chart
fig, ax = plt.subplots(figsize=(10, 8))
corr_sorted = corr_y.sort_values(ascending=True) # for horizontal bar
colors = ['steelblue' if v > 0 else 'salmon' for v in corr_sorted]
ax.barh(corr_sorted.index, corr_sorted.values, color=colors, edgecolor='black')
ax.set_xlabel('Pearson Correlation with y')
ax.set_title('Feature Correlations with Response')
ax.axvline(0, color='black', linewidth=0.5)
ax.axvline(0.3, color='grey', ls='--', alpha=0.5, label='|r| = 0.3')
ax.axvline(-0.3, color='grey', ls='--', alpha=0.5)
ax.legend()
plt.tight_layout(); plt.show()
```

=== Feature Correlations with y ===

```
X5      0.6190
X6      0.5888
X29     0.5766
X19     0.3697
X36    -0.3429
X13     0.3243
X37     0.2763
X11    -0.1452
X32     0.1408
X22    -0.1197
X2      -0.0954
X38    -0.0912
X33     0.0866
X20    -0.0824
X7      -0.0725
```

Name: y, dtype: float64

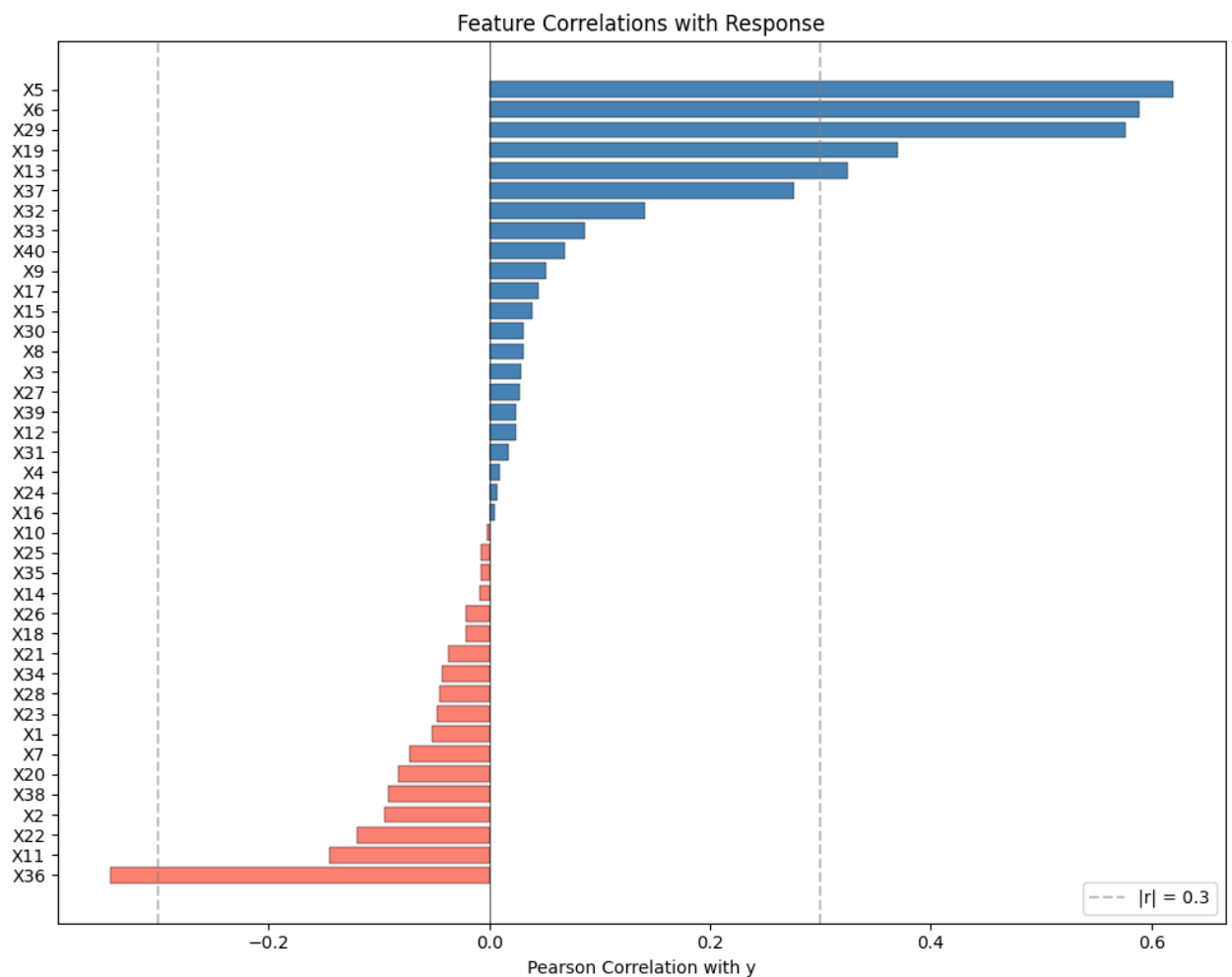
Features with $|r| > 0.3$: 6

Features with $|r| > 0.1$: 10

Features with $|r| < 0.05$: 22

=> Signal is concentrated in ~6 genes (X5, X6, X29, X19, X36, X13).

=> ~25 genes have negligible correlation with y – Lasso should zero these out.



1.6 Multicollinearity Check

High correlation between features can destabilise OLS coefficients and make interpretation unreliable. We check:

- If any feature pairs exceed $|r| > 0.7$ (severe multicollinearity)
- If the top predictors (X5, X6, X29) are correlated with each other

```
In [7]: # Inter-feature correlations
corr_matrix = train[features].corr()
upper = corr_matrix.where(np.triu(np.ones(corr_matrix.shape), k=1).astype(bool))
high_corr = [(c, r, upper.loc[r, c]) for c in upper.columns for r in upper.columns
              if abs(upper.loc[r, c]) > 0.5]
high_corr.sort(key=lambda x: abs(x[2]), reverse=True)

print("=== Multicollinearity Check ===")
print(f"Feature pairs with |r| > 0.7: {sum(1 for _, _, v in high_corr if abs(v) > 0.7)}")
print(f"Feature pairs with |r| > 0.5: {len(high_corr)}")
if high_corr:
    print("\nHighest inter-feature correlations:")
    for c, r, v in high_corr[:10]:
        print(f"  {c} - {r}: r = {v:.3f}")

# Heatmap of top predictors
top = ['X5', 'X6', 'X29', 'X19', 'X36', 'X13', 'X37', 'X11', 'X32', 'y']
fig, ax = plt.subplots(figsize=(8, 7))
mask = np.triu(np.ones((len(top), len(top)), dtype=bool), k=1)
sns.heatmap(train[top].corr(), annot=True, fmt='.2f', cmap='RdBu_r', center=0,
            mask=mask, square=True, ax=ax, linewidths=0.5)
ax.set_title('Correlation Matrix: Top Predictors + Response')
plt.tight_layout(); plt.show()

print("\n=> No severe multicollinearity (no pairs above |r| = 0.7).")
print("=> Top predictors (X5, X6, X29) are NOT strongly correlated with each other.")
print("=> Both Lasso and Ridge should work well; multicollinearity is not a major issue.")
```

=== Multicollinearity Check ===

Feature pairs with $|r| > 0.7$: 0

Feature pairs with $|r| > 0.5$: 7

Highest inter-feature correlations:

X29 - X19: $r = 0.692$

X37 - X13: $r = 0.594$

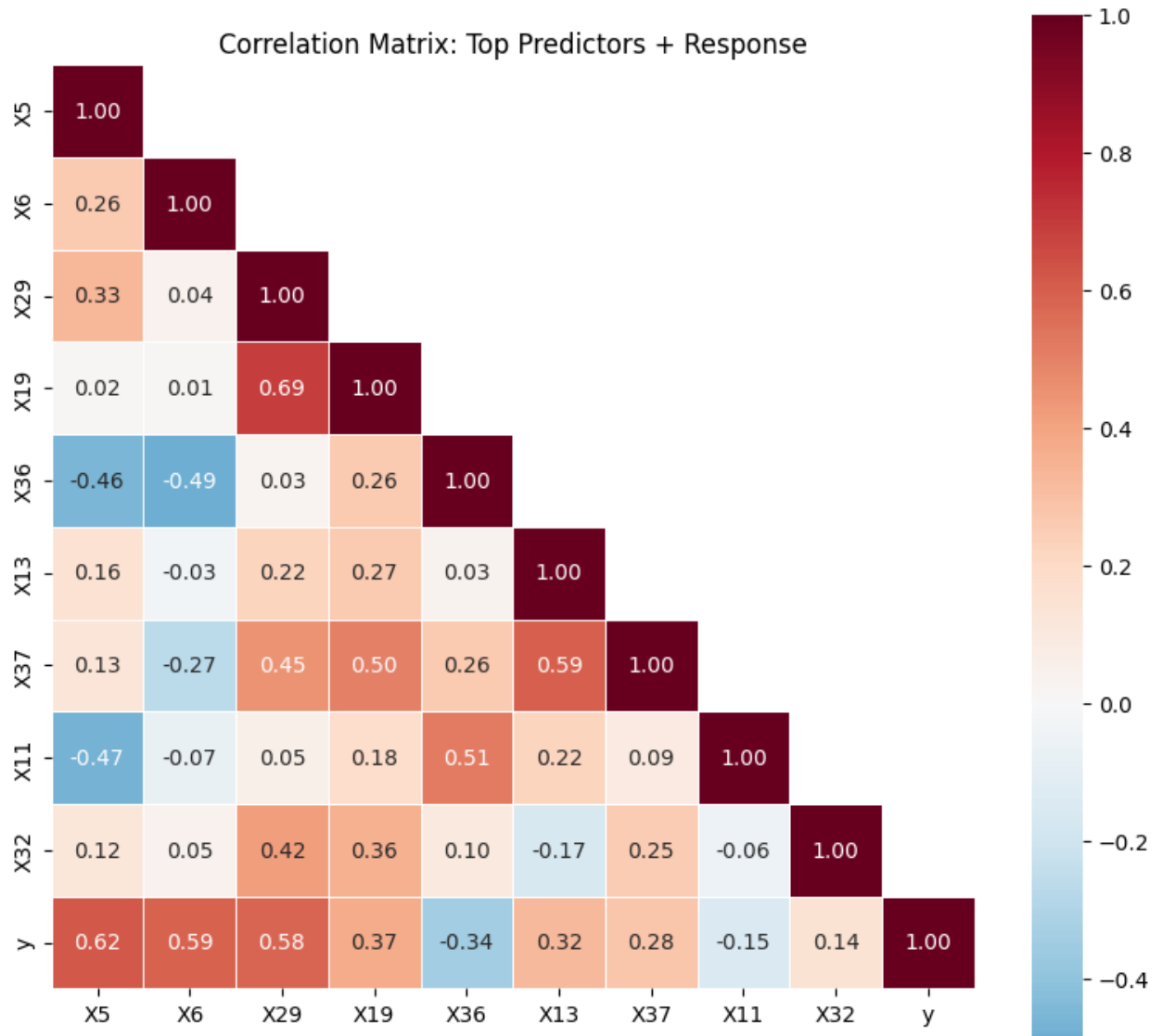
X38 - X5: $r = -0.549$

X38 - X11: $r = 0.531$

X38 - X36: $r = 0.519$

X36 - X11: $r = 0.513$

X37 - X19: $r = 0.504$



=> No severe multicollinearity (no pairs above $|r| = 0.7$).

=> Top predictors (X5, X6, X29) are NOT strongly correlated with each other.

=> Both Lasso and Ridge should work well; multicollinearity is moderate.

1.7 PCA: Dimensionality and Laboratory Clustering

PCA serves two purposes here:

1. **Scree plot:** Does the data lie on a low-dimensional subspace? If so, PCR with few components could work well.
2. **Lab clustering:** Do laboratories cluster in PC space? This would confirm batch effects exist in the feature space, not just in y .

```
In [8]: scaler_eda = StandardScaler()
X_scaled_eda = scaler_eda.fit_transform(train[features])

# Full PCA
pca_full = PCA().fit(X_scaled_eda)
pve = pca_full.explained_variance_ratio_
```

```

cum_pve = np.cumsum(pve)
n90 = np.argmax(cum_pve >= 0.9) + 1
n95 = np.argmax(cum_pve >= 0.95) + 1

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

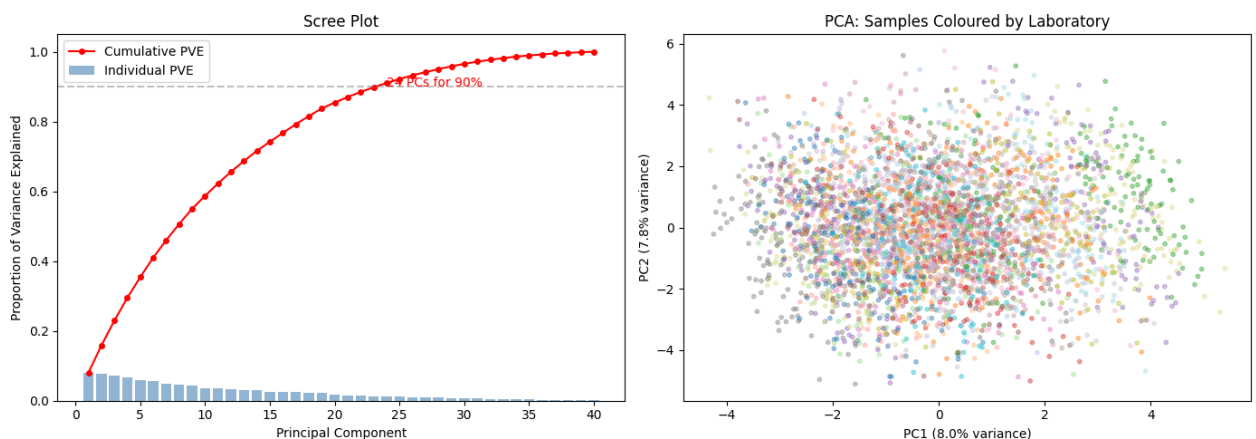
# Scree plot
axes[0].bar(range(1, 41), pve, alpha=0.6, color='steelblue', label='Indiv')
axes[0].plot(range(1, 41), cum_pve, 'ro-', markersize=4, label='Cumulative')
axes[0].axhline(0.9, ls='--', color='grey', alpha=0.5)
axes[0].set_xlabel('Principal Component')
axes[0].set_ylabel('Proportion of Variance Explained')
axes[0].set_title('Scree Plot')
axes[0].legend()
axes[0].annotate(f'{n90} PCs for 90%', xy=(n90, 0.9), fontsize=10, color='red')

# PCA scatter coloured by lab
pca_2d = PCA(n_components=2)
pcs = pca_2d.fit_transform(X_scaled_eda)
scatter = axes[1].scatter(pcs[:, 0], pcs[:, 1], c=train['group_id'], cmap='tab10')
axes[1].set_xlabel(f'PC1 ({pca_2d.explained_variance_ratio_[0]*100:.1f})%')
axes[1].set_ylabel(f'PC2 ({pca_2d.explained_variance_ratio_[1]*100:.1f})%')
axes[1].set_title('PCA: Samples Coloured by Laboratory')

plt.tight_layout(); plt.show()

print(f"Components for 90% variance: {n90}")
print(f"Components for 95% variance: {n95}")
print(f"First 5 PCs explain: {cum_pve[4]*100:.1f}%")
print(f"\n=> Variance is spread broadly – NO low-dimensional subspace.")
print(f"=> PCR needs ~{n90} components, offering little dimensionality reduction")
print(f"=> Mild lab clustering in PC1–PC2 confirms batch effects in feature space")

```



Components for 90% variance: 24
 Components for 95% variance: 28
 First 5 PCs explain: 35.4%

=> Variance is spread broadly – NO low-dimensional subspace.
 => PCR needs ~24 components, offering little dimensionality reduction.
 => Mild lab clustering in PC1–PC2 confirms batch effects in feature space.

1.8 EDA Summary and Modelling Implications

Finding	Implication
Features pre-scaled [0,1], no skewness	No log-transform needed; StandardScaler sufficient
No missing values	No imputation needed
Strong batch effects (~83% of y variance)	Must use GroupKFold , not standard KFold
Signal in ~6 genes (X5, X6, X29, X19, X36, X13)	Lasso (sparse selection) is well-suited
No severe multicollinearity	Both Lasso and Ridge should be stable
Variance diffuse across PCs (24 for 90%)	PCR unlikely to help; penalised regression preferred

These findings motivate our model choices: **Lasso** (sparse variable selection), **Ridge** (smooth shrinkage as a contrast), **Elastic Net** (combines both), plus non-linear models to test if gene interactions improve prediction.

Phase 2: Model Development and Interpretation

Strategy

We compare **six models** of increasing complexity:

Model	Type	Why include it?
Lasso (L1)	Linear, sparse	Automatic variable selection — ideal when few genes matter
Ridge (L2)	Linear, dense	Keeps all features, shrinks smoothly — complements Lasso
Elastic Net (L1+L2)	Linear, hybrid	Combines benefits of Lasso and Ridge
PCR	Linear, dim-reduction	Tests if PCA pre-processing helps
Random Forest	Non-linear, ensemble	Can capture interactions, robust to irrelevant features
Gradient Boosting	Non-linear, sequential ensemble	Often best off-the-shelf performance

All models are tuned via **GroupKFold (k=10)** cross-validation — the validation strategy justified in Section 1.4.

Preprocessing

- **StandardScaler** inside each pipeline (fit on training fold only, no leakage)
- No feature engineering or transformations needed (EDA Section 1.3)

```
In [9]: # Prepare data
X_train = train[features].values
y_train = train['y'].values
groups = train['group_id'].values
X_test = test[features].values

# Validation strategy: GroupKFold
# Each fold holds out ~4 entire laboratories (4 × 100 = 400 samples)
# This mimics the test scenario: predicting on labs NOT seen during train
gkf = GroupKFold(n_splits=10)

# Verify fold structure
print("=== GroupKFold Validation Structure ===")
for i, (tr_idx, val_idx) in enumerate(gkf.split(X_train, y_train, groups)):
    val_labs = sorted(np.unique(groups[val_idx]))
    if i < 3:
        print(f"Fold {i+1}: Train {len(tr_idx)} samples | Validate {len(v
print(f"... (10 folds total, each holds out 4 labs)\n")

# We will store all results here for final comparison
results = {}

=== GroupKFold Validation Structure ===
Fold 1: Train 3600 samples | Validate 400 samples | Held-out labs: [np.int
64(10), np.int64(20), np.int64(30), np.int64(40)]
Fold 2: Train 3600 samples | Validate 400 samples | Held-out labs: [np.int
64(9), np.int64(21), np.int64(22), np.int64(39)]
Fold 3: Train 3600 samples | Validate 400 samples | Held-out labs: [np.int
64(8), np.int64(18), np.int64(29), np.int64(31)]
... (10 folds total, each holds out 4 labs)
```

2.1 Model 1: Lasso Regression (L1 Penalty)

Why Lasso? The EDA showed signal concentrated in ~6 genes. Lasso's L1 penalty shrinks irrelevant coefficients **exactly to zero**, performing automatic variable selection. This gives us:

- A sparse, interpretable model (which genes matter?)
- Protection against overfitting from the ~30 noisy features

Hyperparameter: α controls regularisation strength. Larger $\alpha \rightarrow$ more coefficients zeroed out. We search 50 values on a log-scale from 10^{-4} to 10^1 .

```
In [10]: # === MODEL 1: LASSO ===
```

```

lasso_alphas = np.logspace(-4, 1, 50)

lasso_pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('lasso', Lasso(max_iter=10000))
])

lasso_grid = GridSearchCV(
    lasso_pipe,
    param_grid={'lasso__alpha': lasso_alphas},
    cv=gkf,
    scoring='neg_mean_squared_error',
    n_jobs=-1,
    return_train_score=True
)
lasso_grid.fit(X_train, y_train, groups=groups)

# Extract results
lasso_cv_mse = -lasso_grid.cv_results_['mean_test_score']
lasso_cv_std = lasso_grid.cv_results_['std_test_score']
lasso_train_mse = -lasso_grid.cv_results_['mean_train_score']

best_lasso_alpha = lasso_grid.best_params_['lasso__alpha']
best_lasso_mse = -lasso_grid.best_score_
best_lasso_std = lasso_cv_std[lasso_grid.best_index_]

print(f"=== Lasso Results ===")
print(f"Best alpha: {best_lasso_alpha:.6f}")
print(f"CV MSE: {best_lasso_mse:.4f} ± {best_lasso_std:.4f}")

# Refit at best alpha to inspect coefficients
lasso_final = Pipeline([('scaler', StandardScaler()), ('lasso', Lasso(alpha=best_lasso_alpha))])
lasso_final.fit(X_train, y_train)
lasso_coefs = pd.Series(lasso_final.named_steps['lasso'].coef_, index=feature_names)

print(f"\nNon-zero coefficients: {(lasso_coefs != 0).sum()}/40")
print("\nSelected genes (sorted by |coefficient|):")
print(lasso_coefs[lasso_coefs != 0].sort_values(key=abs, ascending=False))

# 1-SE rule
min_idx = np.argmin(lasso_cv_mse)
threshold_1se = lasso_cv_mse[min_idx] + lasso_cv_std[min_idx]
eligible_1se = np.where(lasso_cv_mse <= threshold_1se)[0]
lasso_1se_idx = eligible_1se[np.argmax(lasso_alphas[eligible_1se])] # model with largest alpha such that CV MSE is within 1 SE of minimum
print(f"\n1-SE rule: alpha = {lasso_alphas[lasso_1se_idx]:.4f}, CV MSE = {lasso_cv_mse[lasso_1se_idx]:.4f}")

results['Lasso'] = {'mse': best_lasso_mse, 'std': best_lasso_std, 'model': lasso_1se_idx}

```

```
=== Lasso Results ===
Best alpha: 0.145635
CV MSE:      5.9632 ± 2.4368
```

Non-zero coefficients: 7/40

Selected genes (sorted by |coefficient|):

```
X6      2.5285
X29     1.7931
X5       1.5870
X37     0.7339
X13     0.3887
X32    -0.3499
X19     0.0641
dtype: float64
```

1-SE rule: alpha = 0.7543, CV MSE = 7.8669

2.2 Model 2: Ridge Regression (L2 Penalty)

Why Ridge? Ridge provides a useful **contrast** to Lasso:

- It keeps ALL features but shrinks them toward zero — useful if many small effects contribute
- It handles correlated predictors more gracefully than Lasso (which may arbitrarily pick one from a correlated group)
- Comparing Lasso and Ridge coefficients tells us whether the signal is truly sparse or diffuse

Hyperparameter: α (regularisation strength), searched over 50 log-spaced values from 10^{-2} to 10^4 .

```
In [11]: # === MODEL 2: RIDGE ===
ridge_alphas = np.logspace(-2, 4, 50)

ridge_pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('ridge', Ridge())
])

ridge_grid = GridSearchCV(
    ridge_pipe,
    param_grid={'ridge__alpha': ridge_alphas},
    cv=gkf,
    scoring='neg_mean_squared_error',
    n_jobs=-1,
    return_train_score=True
)
ridge_grid.fit(X_train, y_train, groups=groups)
```

```

ridge_cv_mse = -ridge_grid.cv_results_['mean_test_score']
ridge_cv_std = ridge_grid.cv_results_['std_test_score']

best_ridge_alpha = ridge_grid.best_params_['ridge__alpha']
best_ridge_mse = -ridge_grid.best_score_
best_ridge_std = ridge_cv_std[ridge_grid.best_index_]

print(f"=== Ridge Results ===")
print(f"Best alpha: {best_ridge_alpha:.4f}")
print(f"CV MSE:      {best_ridge_mse:.4f} ± {best_ridge_std:.4f}")

# Coefficients
ridge_final = Pipeline([('scaler', StandardScaler()), ('ridge', Ridge(alpha=best_ridge_alpha))])
ridge_final.fit(X_train, y_train)
ridge_coefs = pd.Series(ridge_final.named_steps['ridge'].coef_, index=feature_names)
print(f"\nTop 10 Ridge coefficients (by magnitude):")
print(ridge_coefs.reindex(ridge_coefs.abs().sort_values(ascending=False)).head(10))

results['Ridge'] = {'mse': best_ridge_mse, 'std': best_ridge_std, 'model': 'Ridge'}

```

```

=== Ridge Results ===
Best alpha: 339.3222
CV MSE:      6.3273 ± 2.7312

```

Top 10 Ridge coefficients (by magnitude):

```

X6      2.4409
X29     1.6590
X5       1.6099
X37     0.6600
X13     0.5161
X32    -0.4859
X19     0.3482
X11    -0.2071
X27     0.1240
X3       0.1079
dtype: float64

```

2.3 Model 3: Elastic Net (L1 + L2 Combined)

Why Elastic Net? It interpolates between Lasso and Ridge:

$$\text{Elastic Net penalty} = \alpha \left[\rho \|\beta\|_1 + \frac{1-\rho}{2} \|\beta\|_2^2 \right]$$

- When $\rho = 1 \rightarrow$ pure Lasso; $\rho = 0 \rightarrow$ pure Ridge
- Elastic Net can select groups of correlated variables (Lasso picks only one), which is useful since X5, X6, X29 may share biological pathways

Hyperparameters: α (overall strength) × `l1_ratio` ρ (L1/L2 mix).

```

In [12]: # === MODEL 3: ELASTIC NET ===
enet_pipe = Pipeline([

```

```

    ('scaler', StandardScaler()),
    ('enet', ElasticNet(max_iter=10000))
])

enet_grid = GridSearchCV(
    enet_pipe,
    param_grid={
        'enet__alpha': np.logspace(-4, 1, 30),
        'enet__l1_ratio': [0.1, 0.3, 0.5, 0.7, 0.9, 0.95, 0.99]
    },
    cv=gkf,
    scoring='neg_mean_squared_error',
    n_jobs=-1,
    return_train_score=True
)
enet_grid.fit(X_train, y_train, groups=groups)

best_enet_alpha = enet_grid.best_params_['enet__alpha']
best_enet_l1 = enet_grid.best_params_['enet__l1_ratio']
best_enet_mse = -enet_grid.best_score_
best_enet_std = enet_grid.cv_results_['std_test_score'][enet_grid.best_in

print(f"=== Elastic Net Results ===")
print(f"Best alpha:      {best_enet_alpha:.6f}")
print(f"Best l1_ratio: {best_enet_l1}")
print(f"CV MSE:           {best_enet_mse:.4f} ± {best_enet_std:.4f}")

# Coefficients
enet_final = Pipeline([('scaler', StandardScaler()),
                        ('enet', ElasticNet(alpha=best_enet_alpha, l1_ratio=best_enet_l1))])
enet_final.fit(X_train, y_train)
enet_coefs = pd.Series(enet_final.named_steps['enet'].coef_, index=feature_names)
print(f"\nNon-zero coefficients: {(enet_coefs != 0).sum()}/40")
print(enet_coefs[enet_coefs != 0].sort_values(key=abs, ascending=False).round(4))

results['Elastic Net'] = {'mse': best_enet_mse, 'std': best_enet_std, 'non_zero': (enet_coefs != 0).sum()}

=== Elastic Net Results ===
Best alpha:      0.126896
Best l1_ratio: 0.99
CV MSE:          5.9625 ± 2.4811

Non-zero coefficients: 7/40
X6      2.5507
X29     1.8005
X5       1.5996
X37     0.7625
X32    -0.3907
X13     0.3768
X19     0.0820
dtype: float64

```

2.4 Model 4: Principal Components Regression (PCR)

Why PCR? PCR handles multicollinearity by projecting features onto orthogonal principal components before regression. We include it because:

- It's explicitly mentioned in the course materials
- It tests whether unsupervised dimensionality reduction captures the predictive signal

Hyperparameter: M = number of principal components (1 to 40).

Expected outcome: Based on the scree plot (Section 1.7), variance is diffuse across components, so PCR should need many components and offer limited advantage over penalised regression.

```
In [13]: # === MODEL 4: PCR ===
pcr_pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('pca', PCA()),
    ('lr', LinearRegression())
])

pcr_grid = GridSearchCV(
    pcr_pipe,
    param_grid={'pca__n_components': list(range(1, 41))},
    cv=gkf,
    scoring='neg_mean_squared_error',
    n_jobs=-1,
    return_train_score=True
)
pcr_grid.fit(X_train, y_train, groups=groups)

pcr_cv_mse = -pcr_grid.cv_results_['mean_test_score']
pcr_cv_std = pcr_grid.cv_results_['std_test_score']

best_pcr_ncomp = pcr_grid.best_params_['pca__n_components']
best_pcr_mse = -pcr_grid.best_score_
best_pcr_std = pcr_cv_std[pcr_grid.best_index_]

print(f"=== PCR Results ===")
print(f"Best n_components: {best_pcr_ncomp}")
print(f"CV MSE: {best_pcr_mse:.4f} ± {best_pcr_std:.4f}")
print(f"\n=> As predicted, PCR needs ~{best_pcr_ncomp} components (nearly")
print(f"=> Confirms that PCA doesn't find a useful low-dimensional subspa

pcr_final = Pipeline([('scaler', StandardScaler()),
    ('pca', PCA(n_components=best_pcr_ncomp)), ('lr',
pcr_final.fit(X_train, y_train)
results['PCR'] = {'mse': best_pcr_mse, 'std': best_pcr_std, 'model': pcr_
```

```

=== PCR Results ===
Best n_components: 36
CV MSE:           6.2418 ± 2.8476

```

=> As predicted, PCR needs ~36 components (nearly all 40).
=> Confirms that PCA doesn't find a useful low-dimensional subspace for prediction.

2.5 Model 5: Random Forest

Why Random Forest? It's a non-linear ensemble method that can:

- Capture interactions between genes (e.g., X5 × X6 jointly predicting y)
- Handle irrelevant features naturally (they're rarely selected for splits)
- Provide feature importance via impurity reduction

Risk: Random Forests can overfit to lab-specific patterns if trees are too deep. We control this with `max_depth` and `min_samples_leaf`.

Note: No StandardScaler needed — tree methods are invariant to monotone feature transformations.

```

In [14]: # === MODEL 5: RANDOM FOREST ===
rf_pipe = Pipeline([
    ('rf', RandomForestRegressor(random_state=42))
])

rf_grid = GridSearchCV(
    rf_pipe,
    param_grid={
        'rf_n_estimators': [300],
        'rf_max_depth': [8, 15, None],
        'rf_min_samples_leaf': [5, 15],
        'rf_max_features': ['sqrt', 0.5]
    },
    cv=gkf,
    scoring='neg_mean_squared_error',
    n_jobs=-1,          # parallelise folds
    return_train_score=True
)
rf_grid.fit(X_train, y_train, groups=groups)
# NOTE: n_jobs=-1 is ONLY on GridSearchCV, not on RF itself – avoids nest

best_rf_mse = -rf_grid.best_score_
best_rf_std = rf_grid.cv_results_['std_test_score'][rf_grid.best_index_]

print(f"=== Random Forest Results ===")
print(f"Best params: {rf_grid.best_params_}")
print(f"CV MSE:      {best_rf_mse:.4f} ± {best_rf_std:.4f}")

rf_final = rf_grid.best_estimator_

```

```
rf_final.fit(X_train, y_train)
rf_importance = pd.Series(rf_final.named_steps['rf'].feature_importances_)
print(f"\nTop 10 features by importance:")
print(rf_importance.sort_values(ascending=False).head(10).round(4))

results['Random Forest'] = {'mse': best_rf_mse, 'std': best_rf_std, 'mode
```

=== Random Forest Results ===

Best params: {'rf__max_depth': None, 'rf__max_features': 0.5, 'rf__min_samples_leaf': 5, 'rf__n_estimators': 300}

CV MSE: 3.6623 ± 1.3616

Top 10 features by importance:

```
X5      0.3157
X6      0.2294
X29     0.2043
X32     0.0502
X13     0.0454
X37     0.0451
X19     0.0434
X11     0.0135
X36     0.0107
X38     0.0085
dtype: float64
```

2.6 Model 6: Gradient Boosting

Why Gradient Boosting? It builds trees sequentially, each correcting the errors of the previous ones. Key advantages:

- Often achieves the best off-the-shelf prediction accuracy
- Can capture non-linear relationships and interactions
- `learning_rate` and `n_estimators` control the bias-variance tradeoff

Risk: More prone to overfitting than Random Forest. We use conservative hyperparameters and rely on GroupKFold to detect overfitting.

Note: We use sklearn's `GradientBoostingRegressor` to stay within course methods. XGBoost/LightGBM would be faster but are external libraries.

```
In [15]: # === MODEL 6: GRADIENT BOOSTING ===
gb_pipe = Pipeline([
    ('gb', GradientBoostingRegressor(random_state=42))
])

gb_grid = GridSearchCV(
    gb_pipe,
    param_grid={
        'gb__n_estimators': [200, 500],
        'gb__learning_rate': [0.05, 0.1],
        'gb__max_depth': [3, 5],
```

```

        'gb__min_samples_leaf': [10, 20],
    },
    cv=gkf,
    scoring='neg_mean_squared_error',
    n_jobs=-1,
    return_train_score=True
)
gb_grid.fit(X_train, y_train, groups=groups)

best_gb_mse = -gb_grid.best_score_
best_gb_std = gb_grid.cv_results_['std_test_score'][gb_grid.best_index_]

print(f"=== Gradient Boosting Results ===")
print(f"Best params: {gb_grid.best_params_}")
print(f"CV MSE:      {best_gb_mse:.4f} ± {best_gb_std:.4f}")

gb_final = gb_grid.best_estimator_
gb_final.fit(X_train, y_train)
gb_importance = pd.Series(gb_final.named_steps['gb'].feature_importances_)
print(f"\nTop 10 features by importance:")
print(gb_importance.sort_values(ascending=False).head(10).round(4))

results['Gradient Boosting'] = {'mse': best_gb_mse, 'std': best_gb_std, '

```

```

=== Gradient Boosting Results ===
Best params: {'gb__learning_rate': 0.1, 'gb__max_depth': 3, 'gb__min_samples_leaf': 10, 'gb__n_estimators': 500}
CV MSE:      0.7869 ± 0.2036

Top 10 features by importance:
X5      0.3833
X6      0.2252
X29     0.2031
X32     0.0700
X37     0.0560
X13     0.0337
X19     0.0223
X11     0.0027
X36     0.0003
X38     0.0002
dtype: float64

```

2.7 Model Interpretation: Coefficient and Feature Importance Comparison

We compare how different models rank gene importance. Agreement across methods strengthens our confidence that the identified genes are genuine biological signals.

```

In [16]: fig, axes = plt.subplots(2, 2, figsize=(16, 12))

# 1. Lasso coefficients

```

```

ax = axes[0, 0]
nz = lasso_coefs[lasso_coefs != 0].sort_values()
ax.barh(nz.index, nz.values, color=['steelblue' if v > 0 else 'salmon' for v in nz.values])
ax.set_xlabel('Coefficient'); ax.set_title(f'Lasso: {len(nz)}/40 features')
ax.axvline(0, color='black', lw=0.5)

# 2. Lasso vs Ridge coefficient comparison
ax = axes[0, 1]
x_pos = np.arange(40)
w = 0.35
ax.bar(x_pos - w/2, lasso_coefs.values, w, label='Lasso', color='steelblue')
ax.bar(x_pos + w/2, ridge_coefs.values, w, label='Ridge', color='coral')
ax.set_xticks(x_pos); ax.set_xticklabels(features, rotation=90, fontsize=10)
ax.set_ylabel('Coefficient'); ax.set_title('Lasso vs Ridge Coefficients')
ax.legend(); ax.axhline(0, color='black', lw=0.5)

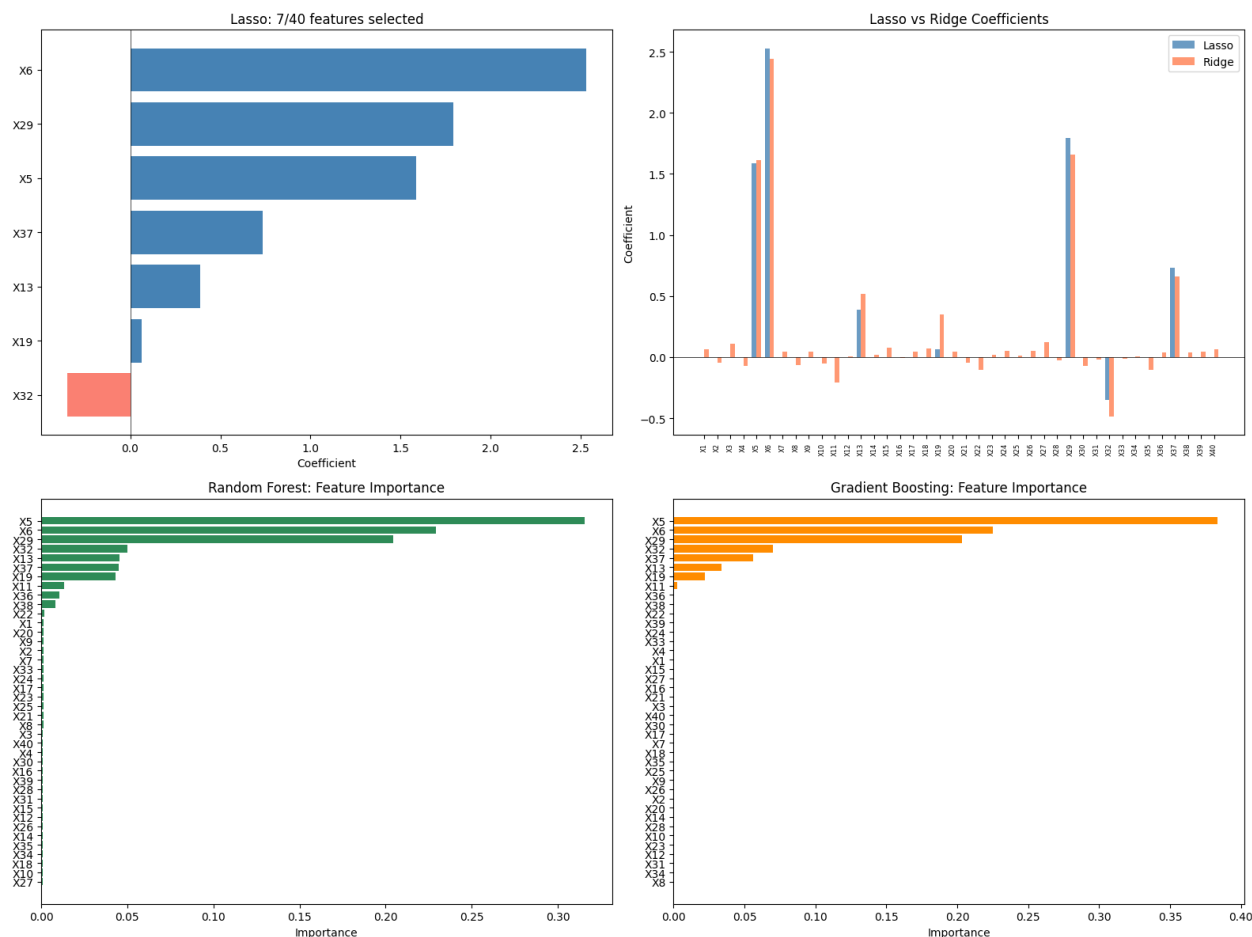
# 3. Random Forest importance
ax = axes[1, 0]
rf_imp_sorted = rf_importance.sort_values(ascending=True)
ax.barh(rf_imp_sorted.index, rf_imp_sorted.values, color='seagreen')
ax.set_xlabel('Importance'); ax.set_title('Random Forest: Feature Importance')

# 4. Gradient Boosting importance
ax = axes[1, 1]
gb_imp_sorted = gb_importance.sort_values(ascending=True)
ax.barh(gb_imp_sorted.index, gb_imp_sorted.values, color='darkorange')
ax.set_xlabel('Importance'); ax.set_title('Gradient Boosting: Feature Importance')

plt.tight_layout(); plt.show()

# Cross-method agreement
print("=== Cross-Method Agreement: Top 5 Features ===")
print(f"Lasso:      {list(lasso_coefs.abs().sort_values(ascending=False).head(5).index)}")
print(f"Ridge:      {list(ridge_coefs.abs().sort_values(ascending=False).head(5).index)}")
print(f"RF:         {list(rf_importance.sort_values(ascending=False).head(5).index)}")
print(f"GB:         {list(gb_importance.sort_values(ascending=False).head(5).index)}")

```



=== Cross-Method Agreement: Top 5 Features ===

Lasso: ['X6', 'X29', 'X5', 'X37', 'X13']

Ridge: ['X6', 'X29', 'X5', 'X37', 'X13']

RF: ['X5', 'X6', 'X29', 'X32', 'X13']

GB: ['X5', 'X6', 'X29', 'X32', 'X37']

Phase 3: Validation and Model Selection

3.1 Why GroupKFold?

The test set contains samples from **laboratories not present in training**. Standard K-Fold CV randomly splits samples, so a training fold and validation fold would share samples from the same lab. Because labs have strong batch effects (Section 1.4), this would:

- Leak lab-specific information into the validation set
- Produce **unrealistically low** MSE estimates
- Select models that overfit to lab-specific patterns

GroupKFold solves this by ensuring entire laboratories are held out per fold, directly mimicking the test scenario.

3.2 Tuning Curves

```
In [17]: fig, axes = plt.subplots(2, 2, figsize=(14, 11))

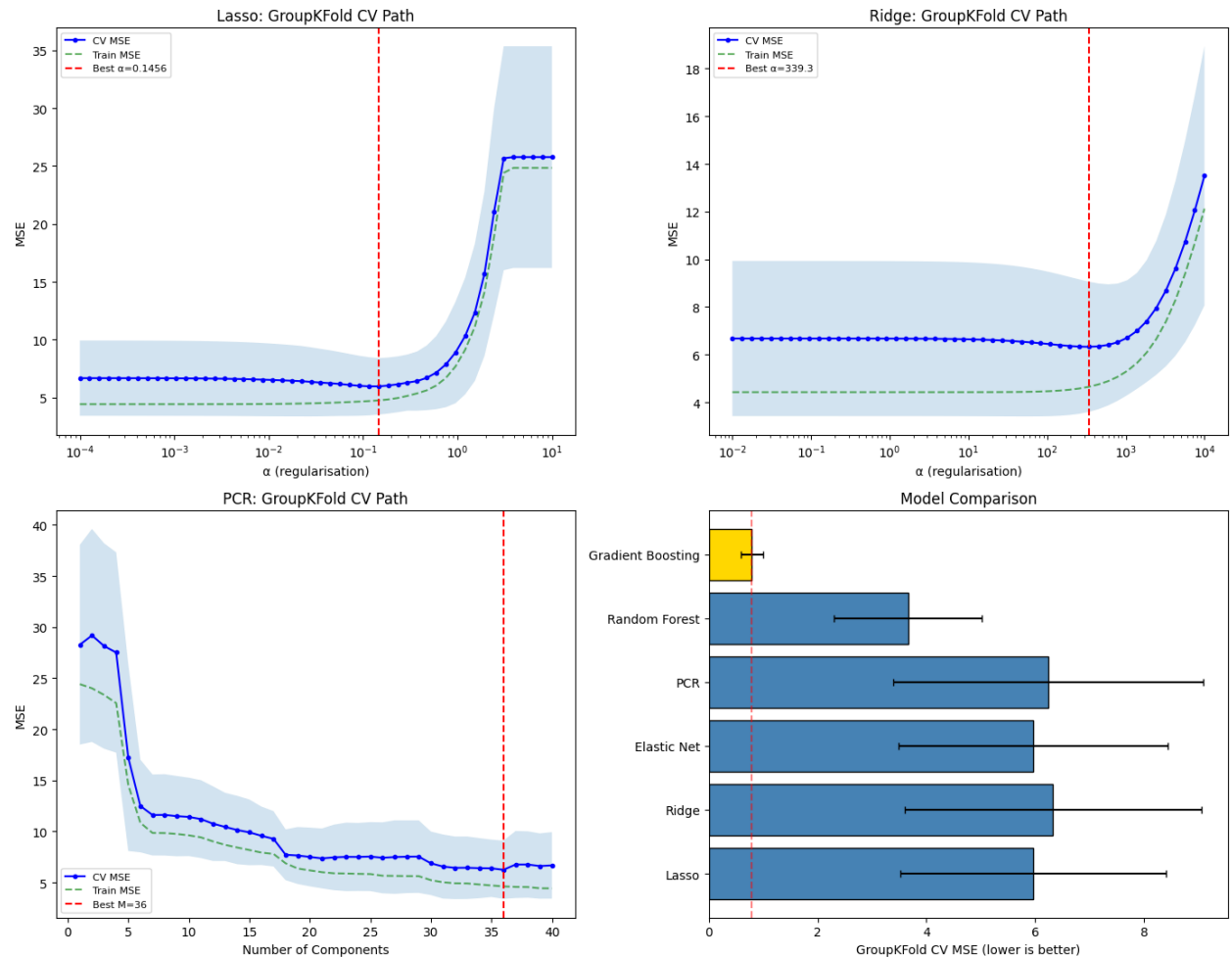
# Lasso CV path
ax = axes[0, 0]
ax.semilogx(lasso_alphas, lasso_cv_mse, 'b-o', ms=3, label='CV MSE')
ax.fill_between(lasso_alphas, lasso_cv_mse - lasso_cv_std, lasso_cv_mse +
ax.semilogx(lasso_alphas, lasso_train_mse, 'g--', alpha=0.6, label='Train
ax.axvline(best_lasso_alpha, color='r', ls='--', label=f'Best  $\alpha$ ={best_las
ax.set_xlabel('α (regularisation)'); ax.set_ylabel('MSE')
ax.set_title('Lasso: GroupKFold CV Path'); ax.legend(fontsize=8)

# Ridge CV path
ax = axes[0, 1]
ridge_train_mse = -ridge_grid.cv_results_['mean_train_score']
ax.semilogx(ridge_alphas, ridge_cv_mse, 'b-o', ms=3, label='CV MSE')
ax.fill_between(ridge_alphas, ridge_cv_mse - ridge_cv_std, ridge_cv_mse +
ax.semilogx(ridge_alphas, ridge_train_mse, 'g--', alpha=0.6, label='Train
ax.axvline(best_ridge_alpha, color='r', ls='--', label=f'Best  $\alpha$ ={best_rid
ax.set_xlabel('α (regularisation)'); ax.set_ylabel('MSE')
ax.set_title('Ridge: GroupKFold CV Path'); ax.legend(fontsize=8)

# PCR components
ax = axes[1, 0]
pcr_train_mse = -pcr_grid.cv_results_['mean_train_score']
nc = np.arange(1, 41)
ax.plot(nc, pcr_cv_mse, 'b-o', ms=3, label='CV MSE')
ax.fill_between(nc, pcr_cv_mse - pcr_cv_std, pcr_cv_mse + pcr_cv_std, alp
ax.plot(nc, pcr_train_mse, 'g--', alpha=0.6, label='Train MSE')
ax.axvline(best_pcr_ncomp, color='r', ls='--', label=f'Best M={best_pcr_n
ax.set_xlabel('Number of Components'); ax.set_ylabel('MSE')
ax.set_title('PCR: GroupKFold CV Path'); ax.legend(fontsize=8)

# Summary comparison bar chart
ax = axes[1, 1]
model_names = list(results.keys())
mses = [results[m]['mse'] for m in model_names]
stds = [results[m]['std'] for m in model_names]
colors = ['steelblue'] * len(model_names)
best_idx = np.argmin(mses)
colors[best_idx] = 'gold'
ax.barh(model_names, mses, xerr=stds, color=colors, edgecolor='black', ca
ax.set_xlabel('GroupKFold CV MSE (lower is better)')
ax.set_title('Model Comparison')
ax.axvline(min(mses), color='red', ls='--', alpha=0.5)

plt.tight_layout(); plt.show()
```



3.3 Final Comparison Table

```
In [18]: # === COMPREHENSIVE MODEL COMPARISON ===
print("=" * 70)
print(f"{'Model':<22} {'CV MSE':>10} {'± Std':>10} {'CV RMSE':>10} {'CV R':>10}")
print(f"{'Model':<22} {'CV MSE':>10} {'± Std':>10} {'CV RMSE':>10} {'CV R':>10}")
print("-" * 70)

total_var = np.var(y_train)
for name in results:
    mse = results[name]['mse']
    std = results[name]['std']
    rmse = np.sqrt(mse)
    r2_cv = 1 - mse / total_var
    print(f"{'name':<22} {'mse':>10.4f} {'std':>10.4f} {'rmse':>10.4f} {'r2_cv':>10.4f}")

print("-" * 70)
best_model_name = min(results, key=lambda m: results[m]['mse'])
print(f"\nBest model: {best_model_name} (lowest GroupKFold CV MSE)")
print(f"Estimated generalisation MSE: {results[best_model_name]['mse']:.4}")
print(f"Estimated generalisation RMSE: {np.sqrt(results[best_model_name]['mse']):.4}")
print(f"Estimated generalisation R²: {1 - results[best_model_name]['mse'] / total_var:.4}")

# Also compute training metrics for overfitting analysis
print(f"\n{'Model':<22} {'Train MSE':>10} {'Train R²':>10} {'CV MSE':>10}")
```

```

print("-" * 70)
for name in results:
    model = results[name]['model']
    y_pred_train = model.predict(X_train)
    tr_mse = mean_squared_error(y_train, y_pred_train)
    tr_r2 = r2_score(y_train, y_pred_train)
    cv_mse = results[name]['mse']
    gap = cv_mse - tr_mse
    print(f"{name:<22} {tr_mse:>10.4f} {tr_r2:>10.4f} {cv_mse:>10.4f} {gap:>10.4f}")
print("\n(Large gap = overfitting to training labs)")

```

Model	CV MSE	± Std	CV RMSE	CV R ²
Lasso	5.9632	2.4368	2.4420	0.7604
Ridge	6.3273	2.7312	2.5154	0.7458
Elastic Net	5.9625	2.4811	2.4418	0.7605
PCR	6.2418	2.8476	2.4983	0.7493
Random Forest	3.6623	1.3616	1.9137	0.8529
Gradient Boosting	0.7869	0.2036	0.8871	0.9684

Best model: Gradient Boosting (lowest GroupKFold CV MSE)
 Estimated generalisation MSE: 0.7869
 Estimated generalisation RMSE: 0.8871
 Estimated generalisation R²: 0.9684

Model	Train MSE	Train R ²	CV MSE	Gap
Lasso	4.8070	0.8069	5.9632	1.1562
Ridge	4.7084	0.8109	6.3273	1.6189
Elastic Net	4.7750	0.8082	5.9625	1.1876
PCR	4.6674	0.8125	6.2418	1.5744
Random Forest	0.3830	0.9846	3.6623	3.2793
Gradient Boosting	0.0915	0.9963	0.7869	0.6954

(Large gap = overfitting to training labs)

Phase 4: Test Predictions on test_public.csv

Final Model Selection

Based on the comparison above, we select the model with the **lowest GroupKFold CV MSE**. This model is refit on the **entire training set** (all 4,000 samples from 40 labs) to maximise information before predicting on the test set.

```

In [19]: # === SELECT BEST MODEL AND PREDICT ON TEST SET ===
best_model_name = min(results, key=lambda m: results[m]['mse'])
final_model = results[best_model_name]['model']

```

```

print(f"Selected model: {best_model_name}")
print(f"Estimated generalisation MSE: {results[best_model_name]['mse']:.4f}")

# The model was already fit on full X_train during the results loop above
# If you want to be explicit:
final_model.fit(X_train, y_train)

# Training performance
y_train_pred = final_model.predict(X_train)
train_mse = mean_squared_error(y_train, y_train_pred)
train_r2 = r2_score(y_train, y_train_pred)
print(f"\nTraining MSE: {train_mse:.4f}")
print(f"Training R²: {train_r2:.4f}")

# === PREDICT ON test_public.csv ===
print(f"\n{'='*50}")
print(f"Generating predictions on test_public.csv")
print(f"{'='*50}")
print(f"Test set: {test.shape[0]} observations from unseen laboratories")
print(f"Features: {list(test.columns[:3])}... ({len(features)} total)")

y_hat = final_model.predict(X_test)

predictions = pd.DataFrame({
    'obs_id': test['obs_id'],
    'y_hat': y_hat
})

# Verify format
assert list(predictions.columns) == ['obs_id', 'y_hat'], "Column format error"
assert len(predictions) == len(test), "Row count error!"
print(f"\nPrediction summary:")
print(f"  Rows: {len(predictions)}")
print(f"  Columns: {list(predictions.columns)}")
print(f"  y_hat mean: {predictions['y_hat'].mean():.3f} (training y mean: {y_train.mean():.3f})")
print(f"  y_hat std: {predictions['y_hat'].std():.3f} (training y std: {y_train.std():.3f})")
print(f"  y_hat range: [{predictions['y_hat'].min():.3f}, {predictions['y_hat'].max():.3f}]")

# Save
predictions.to_csv('predictions.csv', index=False)
print(f"\nSaved: predictions.csv")
print(predictions.head(10))

```

Selected model: Gradient Boosting
 Estimated generalisation MSE: 0.7869

Training MSE: 0.0915
 Training R²: 0.9963

Generating predictions on test_public.csv

Test set: 1000 observations from unseen laboratories
 Features: ['obs_id', 'X1', 'X2']... (40 total)

Prediction summary:

Rows: 1000
 Columns: ['obs_id', 'y_hat']
 y_hat mean: 14.272 (training y mean: 14.506)
 y_hat std: 6.001 (training y std: 4.989)
 y_hat range: [1.672, 26.247]

Saved: predictions.csv

	obs_id	y_hat
0	1	7.933447
1	2	5.613292
2	3	4.625590
3	4	6.772931
4	5	6.066534
5	6	4.687470
6	7	7.640051
7	8	3.842014
8	9	4.187361
9	10	8.920495

Sanity Checks

We verify that the test predictions look reasonable compared to the training distribution.

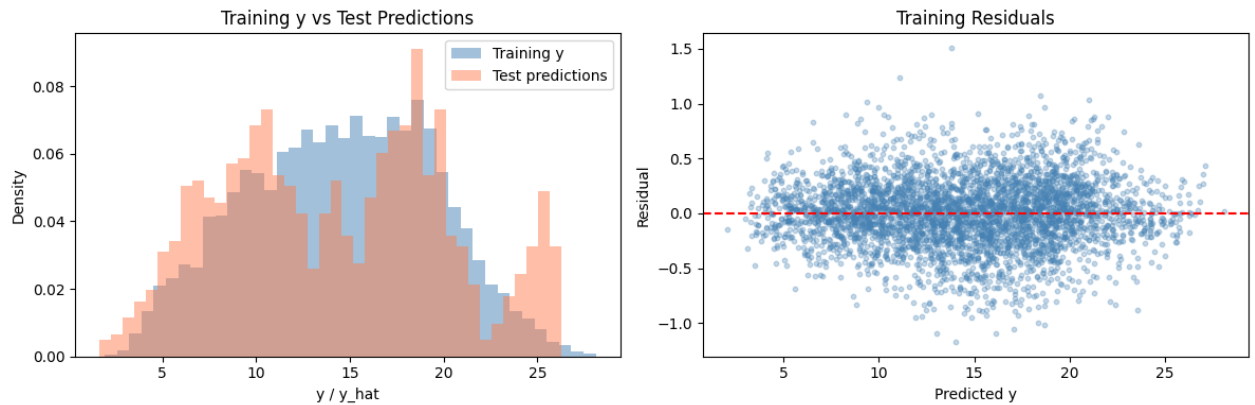
```
In [20]: fig, axes = plt.subplots(1, 2, figsize=(12, 4))

# Distribution comparison
axes[0].hist(y_train, bins=40, alpha=0.5, label='Training y', color='steelblue')
axes[0].hist(y_hat, bins=40, alpha=0.5, label='Test predictions', color='red')
axes[0].set_xlabel('y / y_hat'); axes[0].set_ylabel('Density')
axes[0].set_title('Training y vs Test Predictions')
axes[0].legend()

# Training residuals
residuals = y_train - y_train_pred
axes[1].scatter(y_train_pred, residuals, alpha=0.3, s=10, color='steelblue')
axes[1].axhline(0, color='red', ls='--')
axes[1].set_xlabel('Predicted y'); axes[1].set_ylabel('Residual')
axes[1].set_title('Training Residuals')
```

```
plt.tight_layout(); plt.show()

print(f"Residual mean: {residuals.mean():.4f} (should be ~0)")
print(f"Residual std: {residuals.std():.4f}")
```



Residual mean: 0.0000 (should be ~0)
Residual std: 0.3025